

Document Intelligence

An engineering research report on Layout-aware field + table extraction

Sai Veda

Independent Project Research

2025

Contents

1. Abstract	3
2. Introduction	3
2.1 Research Problem	3
2.2 Research Questions and Hypotheses	3
3. Technical Background Review	4
3.1 Design Foundations	4
3.2 Supporting Examples	4
4. Research Method	4
5. Data Description	5
6. Analysis	5
6.1 Benchmark and Validation Results	5
6.2 Test Result Screenshots	5
6.3 Validation Checklist	6
7. Discussion	6
8. Conclusion	7
9. Future Work	7
10. References	7

1. Abstract

This project documents a layout-aware document AI stack that treats geometry as a primary signal, enabling structured field extraction, table recovery, routing, and search over heterogeneous business documents. The system shows that strong field extraction and routing can be demonstrated credibly without a full OCR or vision stack, as long as the token-plus-box representation and reconstruction logic are engineered carefully. The implementation evidence is drawn from committed repository artefacts, benchmark outputs, automated tests, and generated screenshots.

Keywords: Layout-aware field + table extraction, implementation evidence, benchmark results, project validation

2. Introduction

Document understanding fails when text is stripped from layout. Invoices, receipts, and forms depend on where a token sits on the page, not only on the token itself, so the pipeline was designed around spatial context from the start. Layout-aware document understanding for invoices, receipts, purchase orders and forms - at scale. Turn a page of tokens-with-boxes into structured, typed fields: identifiers, dates, parties, totals, and the full line-item table - then route the document and make its content searchable. Fully offline, deterministic, no GPU, no paid APIs. Measured on a held-out set of 2,000 documents / 98,687 tokens (train: 8,000 docs / 396,586 tokens), seed-fixed and reproducible with make run:

2.1 Research Problem

Document understanding fails when text is stripped from layout. Invoices, receipts, and forms depend on where a token sits on the page, not only on the token itself, so the pipeline was designed around spatial context from the start.

2.2 Research Questions and Hypotheses

Research question: Does the implemented project address the stated technical problem in a complete and reviewable manner?

Hypothesis: The repository design, architecture notes, and implementation artefacts are expected to demonstrate end-to-end coverage of the core project objective.

Research question: Do the benchmark and test artefacts support the main performance or quality claim?

Hypothesis: The recorded evidence is expected to support the headline result reported for this project: field macro-F1 0.976; 100k docs, doc-type 100%.

Research question: Can the results be reviewed and reproduced from committed repository evidence?

Hypothesis: The presence of 18 automated tests, benchmark outputs, and generated screenshots is expected to provide a transparent validation path.

3. Technical Background Review

3.1 Design Foundations

- 1 Spatial context is treated as load-bearing because field meaning depends heavily on page position. The pipeline stays spatial from the first token box to the recovered fields and tables, which is why the outputs stay structured.
- 2 Synthetic rendered layouts keep labels exact while still reflecting realistic document structure.
- 3 The pipeline is built for structured outputs, not just page-level classification scores.

3.2 Supporting Examples

- Rendered-layout document generation with gold token boxes.
- Layout-aware feature pipeline and token classification.
- Key-value pairing, table extraction, routing, and search hooks.

4. Research Method

The project was reviewed as an engineering artefact using repository documentation, architecture notes, benchmark evidence, automated tests, and generated screenshots. Generated structured pages with gold token positions so every downstream metric could be trusted. Built token-level classification that respects geometry rather than flattening the page to plain text. Grouped fields into key-value pairs and line-item tables using spatial

logic. Validated extraction quality, routing, and search-readiness on held-out document sets.

5. Data Description

Source: committed repository artefacts including implementation code, automated tests, benchmark outputs, architecture notes, and generated visual evidence. Coverage: 18 automated tests, 1 benchmark table(s), and 4 screenshot asset(s) were available for document preparation. Schema / artefact types: README overview, ARCHITECTURE design note, benchmark markdown and CSV outputs, test suites, and figure assets generated from the project run path. Availability: all evidence cited in this document is sourced from the repository itself.

6. Analysis

6.1 Benchmark and Validation Results

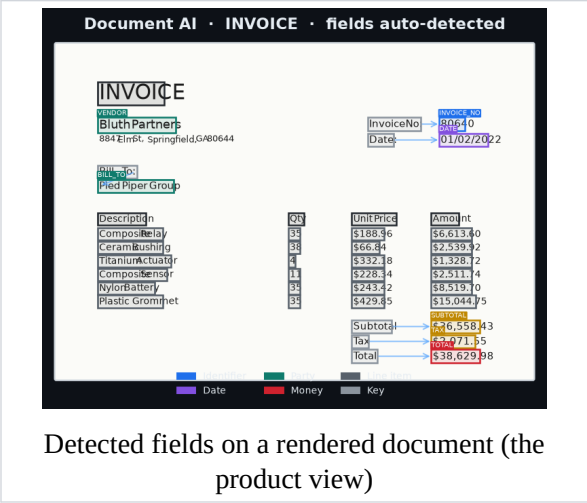
The headline repository result is reported as: field macro-F1 0.976; 100k docs, doc-type 100%. The table below reproduces benchmark evidence included in the repository where available.

Docs	Tokens	Gen docs/s	Gen tokens/s	Infer docs/s	Infer tokens/s
2,000	99,071	728	36,065	1,838	90,840
10,000	497,136	1,027	51,037	2,262	111,818
50,000	2,482,852	774	38,452	2,108	104,170
100,000	4,964,026	1,009	50,080	2,405	118,876

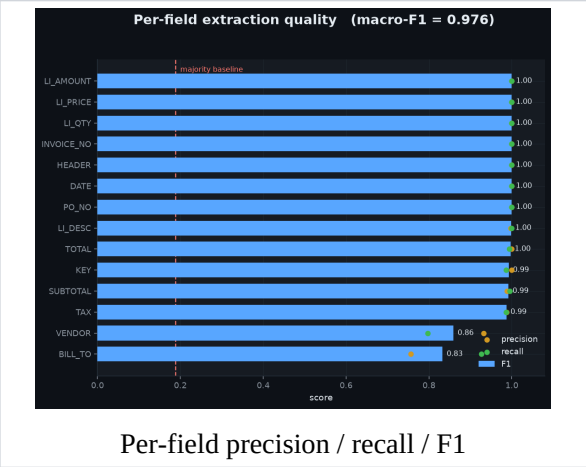
Field-level macro-F1 is strong because the model has access to both text and layout information. The screenshots are useful because they show recovery of tables and key-value relationships, not only summary metrics. Routing quality matters operationally because extraction and downstream indexing share the same document understanding layer.

6.2 Test Result Screenshots

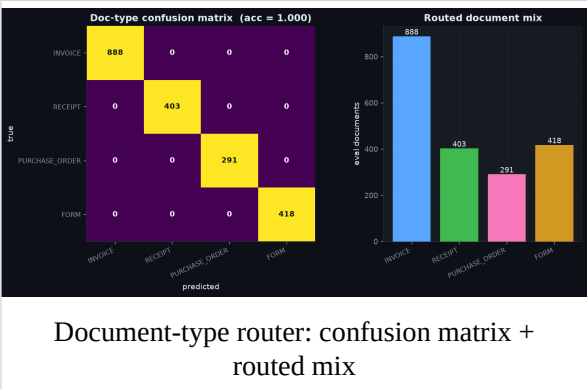
The following figures are drawn directly from the repository screenshot assets and are included as visual evidence of measured outputs, dashboards, or generated validation artefacts.



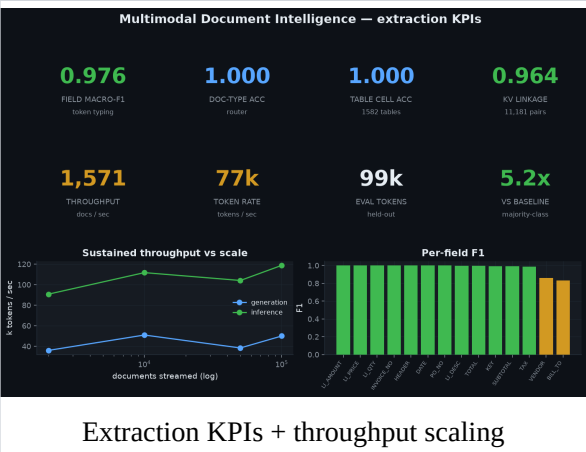
Detected fields on a rendered document (the product view)



Per-field precision / recall / F1



Document-type router: confusion matrix + routed mix



Extraction KPIs + throughput scaling

6.3 Validation Checklist

Item	Status	Evidence
Automated tests	Available	18 tests committed in repository validation flow.
Benchmark results	Available	Benchmark markdown tables and result summaries were reviewed.
Screenshots	Available	4 screenshot asset(s) included in the repository evidence set.
Architecture note	Available	ARCHITECTURE.md documents design choices, scale assumptions, and trade-offs.

7. Discussion

The system shows that strong field extraction and routing can be demonstrated credibly without a full OCR or vision stack, as long as the token-plus-box representation and reconstruction logic are engineered carefully. Field-level macro-F1 is strong because the

model has access to both text and layout information. The screenshots are useful because they show recovery of tables and key-value relationships, not only summary metrics. Routing quality matters operationally because extraction and downstream indexing share the same document understanding layer. The analysis indicates that the repository is strongest where implementation, benchmark artefacts, and screenshots point to the same conclusion.

8. Conclusion

This project document concludes that Document Intelligence is best understood as a complete technical implementation supported by repository-native evidence. This project documents a layout-aware document AI stack that treats geometry as a primary signal, enabling structured field extraction, table recovery, routing, and search over heterogeneous business documents.

9. Future Work

- Add scanned-document or OCR-noise simulation to stress the layout pipeline further.
- Support more document families and more complex nested table structures.
- Connect extracted entities to downstream review and retrieval workflows.

10. References

1. README.md - primary repository overview and headline results.
2. ARCHITECTURE.md - system design, implementation rationale, and scale notes.
3. benchmarks/results.csv - benchmark or result artefact used in the analysis section.
4. benchmarks/results.md - benchmark or result artefact used in the analysis section.
5. tests/ - automated validation suite used to support implementation claims.
6. assets/dashboard.png - generated screenshot evidence included in the report.
7. assets/detected_document.png - generated screenshot evidence included in the report.